Docker Compose: Multi-Container Applications

Pertemuan 5 • Cloud Computing • 90 menit

Dari satu container menuju aplikasi multi-service yang dapat dijalankan dengan satu perintah.

Target praktikum

Menjalankan web application + PostgreSQL menggunakan Docker Compose.

Capaian Pembelajaran

- Menjelaskan kebutuhan multi-container application.
- Memahami service, image/build, port, network, volume, dan environment.
- Menulis file compose.yaml yang valid.
- Menjalankan web app dan database secara bersamaan.
- Memeriksa logs, konektivitas, persistence, dan lifecycle.

Review Pertemuan 4: Docker

Image

Template read-only untuk membuat container.

Container

Instance aplikasi yang berjalan.

Dockerfile

Instruksi untuk membangun image.

Pertanyaan: bagaimana jika aplikasi membutuhkan database, cache, dan worker?

Masalah: Aplikasi Tidak Hanya Satu Container

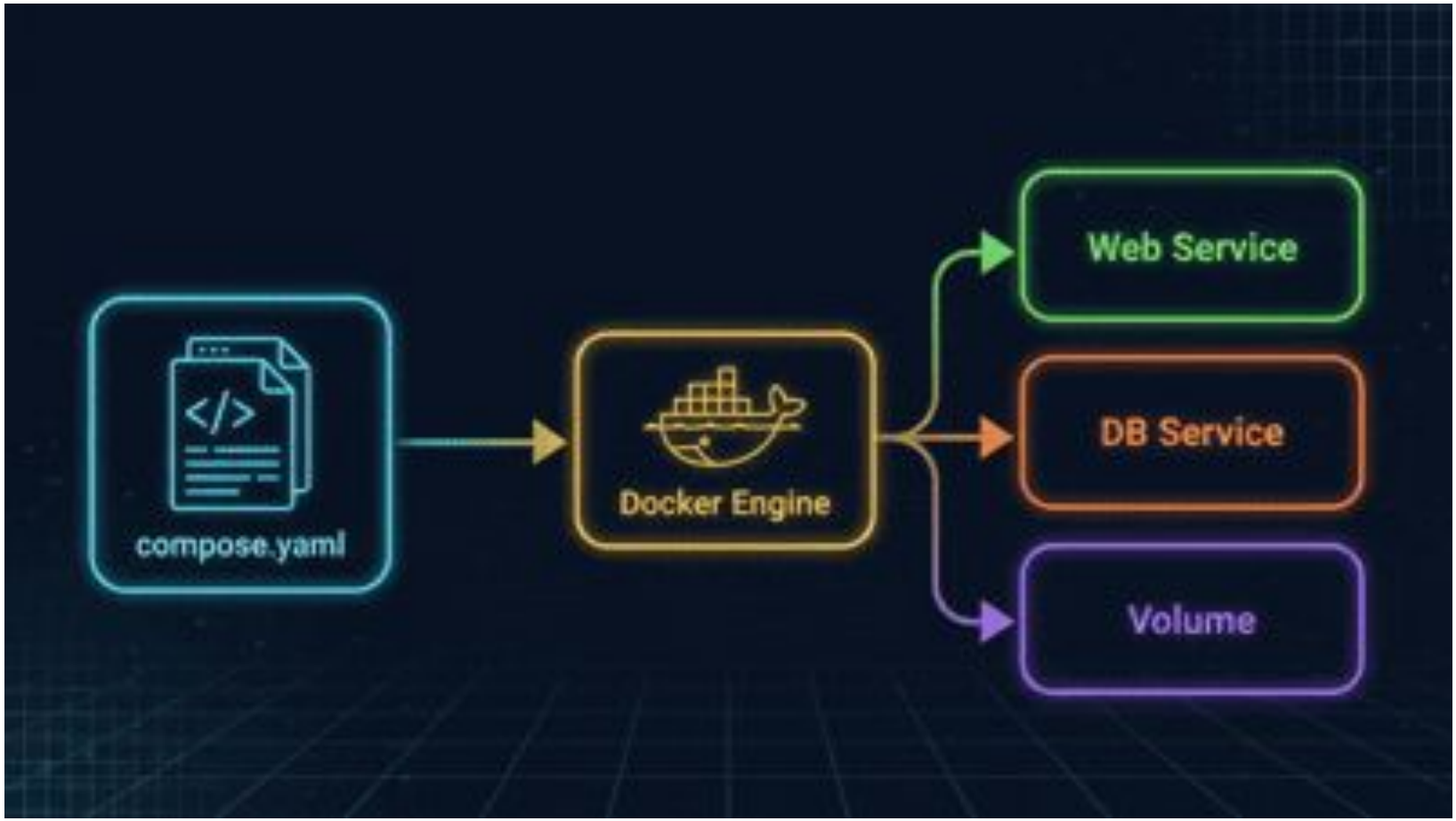


Tantangan: startup, network, environment, volume, dan koordinasi antarlayanan.

Apa Itu Docker Compose?

- Tool untuk mendefinisikan dan menjalankan aplikasi multi-container.
- Konfigurasi disimpan dalam satu file YAML (compose.yaml).
- Setiap komponen aplikasi didefinisikan sebagai service.
- Satu command dapat membuat network, volume, dan container.
- Sangat cocok untuk development, testing, dan lab lokal.

Arsitektur Docker Compose



Struktur Dasar compose.yaml

```yaml
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"

  db:
    image: postgres:16-alpine
```

Konsep 1: Services

Service web

Menjalankan aplikasi web.

Bisa menggunakan image atau build.

Dapat mengekspos port ke host.

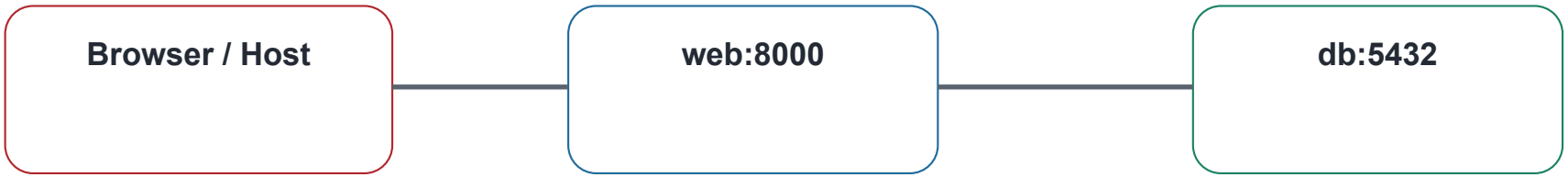
Service db

Menjalankan database.

Umumnya tidak perlu port publik.

Membutuhkan persistent volume.

Konsep 2: Docker Compose Network



Service discovery

Di dalam Compose network, web dapat menghubungi database dengan hostname db (bukan localhost).

Konsep 3: Port Mapping

```
services:
  web:
    ports:
      - "8000:8000"  # host:container

# Browser host: http://localhost:8000
# Antar-container: http://web:8000
```

Database tidak perlu diekspos ke host kecuali ada kebutuhan lab khusus.

Konsep 4: Environment Variables

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: cloudlab
      POSTGRES_USER: clouduser
      POSTGRES_PASSWORD: ${DB_PASSWORD}
```

Untuk lab: gunakan file .env lokal; jangan commit password ke GitHub.

Konsep 5: Persistent Volumes



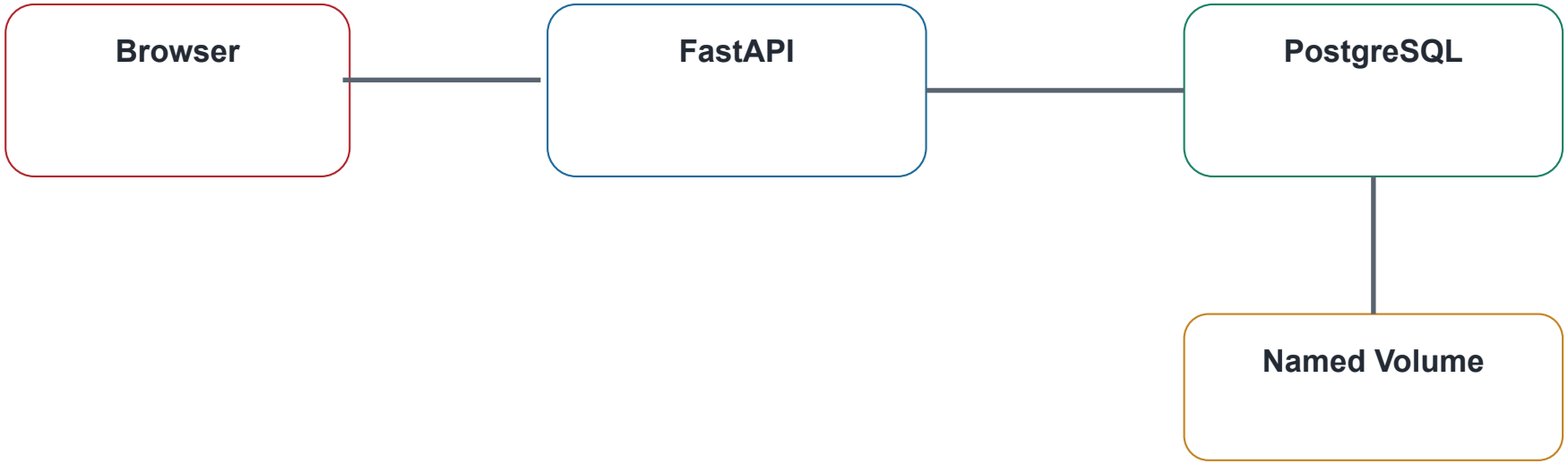
Mengapa volume?

Data tetap ada walaupun container dihentikan atau dibuat ulang. Hindari docker compose down -v jika ingin mempertahankan data.

Konsep 6: Depends_on & Healthcheck

- depends_on mengatur dependency/startup ordering.
- Startup container bukan jaminan aplikasi sudah siap menerima koneksi.
- Healthcheck memeriksa kesiapan layanan.
- Gunakan condition: service_healthy ketika service memiliki healthcheck.
- Aplikasi tetap perlu retry ketika dependency terputus.

Arsitektur Lab: Web + PostgreSQL



Lab ini dapat dijalankan secara lokal tanpa akun cloud atau kartu kredit.

Persiapan Praktikum (5 menit)

```
docker --version
docker compose version
mkdir cloud-compose-lab
cd cloud-compose-lab
mkdir app
```

Pastikan Docker Engine aktif sebelum memulai.

Struktur Folder Proyek

```
cloud-compose-lab/
├── app/
│   ├── main.py
│   └── requirements.txt
├── Dockerfile
├── compose.yaml
└── .env
```

Langkah 1: Aplikasi FastAPI

```
# app/main.py
import os
from fastapi import FastAPI
import psycopg

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Compose Lab OK"}
```

Endpoint awal digunakan untuk menguji web service sebelum koneksi database.

Langkah 2: Requirements & Dockerfile

```
# app/requirements.txt
fastapi
uvicorn
psycopg[binary]

# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Langkah 3: Compose File (Bagian 1)

```
services:
  web:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://clouduser:${DB_PASSWORD}@db:5432/cloudlab
    depends_on:
      db:
        condition: service_healthy
```

Langkah 4: Compose File (Bagian 2)

```
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: cloudlab
      POSTGRES_USER: clouduser
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U clouduser -d cloudlab"]
      interval: 5s
      retries: 10

volumes:
  postgres_data:
```

Langkah 5: File .env

```
# .env (hanya untuk lab lokal)
DB_PASSWORD=cloudlab123

# Tambahkan ke .gitignore:
.env
```

Security note

Jangan menggunakan password contoh ini untuk production.

Langkah 6: Build & Start

```
docker compose config
docker compose up -d --build
docker compose ps

# Uji dari browser:
# http://localhost:8000
```

Ekspektasi: web dan db berjalan; endpoint mengembalikan JSON.

Langkah 7: Test Database Connection

```
# Dari host, jalankan query di dalam db container:
docker compose exec db psql -U clouduser -d cloudlab

# Pada prompt psql:
SELECT current_database();
SELECT now();
\q
```

Opsional: perluas endpoint FastAPI agar membaca hasil query database.

Lifecycle: Start, Stop, Restart, Down

```
docker compose ps
docker compose logs -f web
docker compose restart web
docker compose stop
docker compose start
docker compose down

# HATI-HATI: menghapus named volume
docker compose down -v
```

Troubleshooting Cepat

Port sudah dipakai

Ganti host port: 8001:8000.

Database belum siap

Periksa healthcheck dan logs.

Hostname salah

Gunakan db, bukan localhost, dari container web.

Data hilang

Periksa volume dan hindari down -v.

Latihan Kelompok: Uji Persistence

- Jalankan web dan database menggunakan Compose.
- Buat tabel sederhana di PostgreSQL.
- Masukkan satu record pengujian.
- Jalankan docker compose down (tanpa -v).
- Jalankan docker compose up -d.
- Verifikasi record masih ada dan jelaskan alasannya.

Mini Quiz & Evaluasi

- Apa beda image dan service pada Compose?
- Mengapa web memakai hostname db untuk mengakses database?
- Apa beda ports dan expose secara konsep?
- Mengapa database memerlukan named volume?
- Apa risiko docker compose down -v?
- Mengapa depends_on saja tidak menjamin database siap?

Ringkasan

- Compose menyederhanakan pengelolaan aplikasi multi-container.
- Komponen penting: services, networks, volumes, environment, healthchecks.
- Praktik: FastAPI + PostgreSQL berjalan dengan satu perintah.